NRLC-06: SLO & Workload Migration

> **Series:** NRLC | **Notebook:** 6 of 9 | **Created:** April 2026 | **Last Updated:** 04/17/2026

Overview

SLOs are the service contracts that survive migration only if their math survives. Workloads are the groupings that determine what an SLO targets. This deep dive covers SLO metric expression migration, the SLO auditor that validates math equivalence, and the conversion of NR workloads to Gen3 OpenPipeline enrichments + IAM-scoped buckets.

Phase 23 key transactions: `key_transaction_transformer` auto-emits a bundle for each NR Key Transaction — `builtin:monitoring.slo` definition + OpenPipeline enrichment attribute (`keyTransaction.name`) + Workflow with `migratedFrom` tag. Phase 11 `slo_transformer` handles SLO v1 / v2; Phase 11 `workload_transformer` emits `builtin:segment` + bucket-scoped IAM policy (Gen3 default) with `LegacyWorkloadTransformer` preserving Management Zone under `--legacy` .

Table of Contents

Prerequisites

Requirement	Details
Audience	SRE leads, service owners maintaining SLOs
Standalone	This notebook is self-contained for SLO + workload migration. No required prerequisite reading.
Optional depth	NRLC-02 (full NRQL→DQL compiler), NRLC-08 (validation), NRLC-09 (toolchain)
Stakes	SLOs drive SLA reporting; arithmetic mismatch causes compliance issues

Embedded Translation Context — SLO Indicator Patterns

Every SLO contains a NRQL indicator query that must produce the equivalent DQL. The patterns below cover the vast majority of SLO indicators.

Availability SLO

```
-- NRQL: "good" events / total
SELECT percentage(count(*), WHERE error IS FALSE) FROM Transaction
-- target: &gt; 99.9
```

```
-- DQL (SLO metric expression)
100.0 * countIf(isNull(error)) / count()
-- target: 99.9
```

Latency SLO

```
-- NRQL
SELECT percentage(count(*), WHERE duration &lt; 0.5) FROM Transaction
-- target: &gt; 99
```

```
-- DQL
100.0 * countIf(duration &lt; duration("500ms")) / count()
-- target: 99
```

Error budget burn rate

```
-- NRQL
SELECT count(*) FROM TransactionError SINCE 1 hour ago
-- threshold: above (1 - SLO_target) * total
```

```
-- DQL (in SLO definition)
100.0 * countIf(isNotNull(error)) / count()
-- target inverted: error budget = 100 - SLO target
```

Translation Confidence for SLOs

Indicator pattern	Confidence	Notes
Percentage-based SLI	HIGH	Direct
Threshold-based with `duration`	HIGH	Mind the unit (`duration("500ms")` vs ms numeric)
Error budget burn	MEDIUM	Verify alignment of denominator
Multi-source SLI (joined data)	LOW	Manual review

1. SLO Models Compared

NR SLO Model

An NR SLO has:
- `target` (e.g., 99.9%)
- `timeWindow` (rolling 7d, 28d, etc.)
- `indicator` (a NRQL query computing the SLI value)
- `threshold` (the value the SLI must satisfy)
- entities the SLO targets (via workload or entity selector)

DT SLO Model (v2)

An DT SLO has:
- `name`, `description`
- `target` (numeric percentage)
- `warning` threshold (early-warning percentage)
- `evaluation` window (rolling)
- `metric expression` (DQL or metric query)
- `filter` (entity selector or DQL attribute filter; bucket + enriched-attribute scope in Gen3)

Structurally similar; the math layer is what migrates.

2. Metric Expression Migration

The SLI is a fraction:

```
SLI = good_events / total_events
```

Both NR and DT model this; the difference is how each side counts.

Example — Availability SLO

NR NRQL indicator:

```
SELECT count(*) FROM Transaction WHERE error IS FALSE
```

NR threshold: result / total > 0.999

DT SLO metric expression (DQL):

```
100.0 * countIf(isNull(error)) / count()
```

DT target: 99.9

Example — Latency SLO

NR NRQL indicator:

```
SELECT percentage(count(*), WHERE duration &lt; 0.5) FROM Transaction
```

DT SLO metric expression:

```
100.0 * countIf(duration &lt; duration("500ms")) / count()
```

Conversion considerations:

- NR units in NRQL are often seconds; DT durations require explicit units (`duration("500ms")` or numeric ns)
- NR's `error IS FALSE` (boolean) maps to DT's `isNull(error)` or `not isNotNull(error)` depending on schema
- Time windows: NR `SINCE 7 days ago` becomes DT `evaluationWindowMinutes: 10080`
- Entity scope: NR's workload becomes a DT bucket + enriched-attribute filter (Gen3 pattern)

3. SLO Auditor

The `Dynatrace-NewRelic` project includes an **SLO auditor** (`registry/SLOAuditor`) that validates SLO math equivalence:

1. For each migrated SLO, run the original NRQL against NR — capture SLI value.
2. Run the converted DQL against DT for the same window — capture SLI value.
3. Compute delta. Pass if within configured tolerance (defaults set inside the auditor).
4. Fail if delta exceeds tolerance — emit a diff report showing per-window values.

```
python3 migrate.py audit-slos
```

> Note: the window and tolerance are set inside the auditor configuration, not via CLI flags. Tune those in the project's audit config rather than on the command line.

Output:

```
SLO: Checkout Availability
  NR SLI:  99.94%
  DT SLI:  99.92%
  Delta:    0.02%  ✅ PASS

SLO: Search Latency p95
  NR SLI:  98.10%
  DT SLI:  97.20%
  Delta:    0.90%  ❌ FAIL (exceeds configured tolerance)
  → see slo-diff-report.json for per-window analysis
```

4. Workload → OpenPipeline Enrichment + IAM Policy

NR workloads are user-defined entity collections, scoped by entity tags or explicit GUID lists.

In **Dynatrace Gen3**, the equivalent is built on **OpenPipeline enrichment** — ingested data is enriched with workload-identifying attributes at ingest (e.g., `workload.name`, `team.owner`, `environment`), and **IAM policies + bucket routing** scope access by those enriched attributes. This is fundamentally different from the Gen2 management-zone or auto-tagging approach (both of which scope entity *visibility*); the Gen3 model scopes the *data itself* via attributes that travel with every record.

NR Workload Field	Gen3 Equivalent
`name`	OpenPipeline enrichment value (e.g., `workload.name = "checkout"`)
`entityGuids[]`	OpenPipeline matcher resolves the entity by name/host group and emits the enriched attribute
`entitySearchQueries[]` (NRQL filter)	OpenPipeline matcher conditions (matchesValue on namespace, host group, etc.)
Tags-based selection	OpenPipeline matcher reads existing tags and emits a normalized workload attribute

The `WorkloadTransformer` emits one OpenPipeline enrichment rule per workload (plus an optional bucket-routing rule if the workload should land in its own bucket) and a recommended IAM policy condition (e.g., `storage:bucket-name == "workload_checkout_logs"`). Once enriched, the workload identifier is queryable in DQL (`filter workload.name == "checkout"`) and durable across dashboards, alerts, and SLOs without depending on Gen2 entity grouping.

5. Entity Selector Translation

DT entity selectors are the language for entity rules. Common patterns:

NR Concept	DT Entity Selector
All hosts in account	`type(HOST)`
Hosts tagged `env:prod`	`type(HOST),tag("env:prod")`
Specific service by name	`type(SERVICE),entityName("checkout-service")`
Services in a namespace	`type(SERVICE),fromRelationships.runsOn(type(KUBERNETES_NAMESPACE),entityName("prod"))`
All entities in a workload	`tag("workload:<name>")` (Gen2 entity tag) **or** Gen3 DQL filter `filter workload.name == "<name>"` (attribute set by OpenPipeline enrichment)

Recommendation: prefer **entity selectors over GUIDs** in any migrated artifact (dashboards, alerts, SLOs). Selectors are portable across tenants; GUIDs are not.

6. Validation — 7-Day SLI Delta

Run the SLO auditor over a 7-day window. Acceptance criteria:

- Per-SLO delta ≤ 0.5% (tunable per service-criticality)
- All SLOs report a value (no MISSING_DATA results)
- Bucket + enriched-attribute scope returns the same entity count in both platforms (±5%)

When delta exceeds tolerance:

1. Inspect the per-window diff report — is the delta consistent (suggests a math bug) or sporadic (suggests data-availability issues)?
2. Check unit conversion (ms vs. s vs. ns)
3. Check entity scope — is the bucket / OpenPipeline enrichment filter capturing the same set as NR's workload?
4. Check time alignment — NR's `SINCE 7 days ago` may bin differently than DT's `from:-7d`

Summary

SLO migration succeeds when the math is equivalent within tolerance. The auditor automates this validation; workload → OpenPipeline enrichment conversion is mechanical for tag-based workloads and manual for GUID-based ones. Always run the 7-day delta check before declaring SLO migration complete.

Continue to **NRLC-07 Logs, Tags & Drop Rules**.

Tooling for SLO-Only Migration

```
# 1. Inventory NR SLOs and Workloads
python3 migrate.py migrate --export-only --components slos,workloads --output ./slo-export

# 2. Translate metric expressions; emit SLO + OpenPipeline enrichment definitions
python3 migrate.py migrate --transform-only --components slos,workloads --report

# 3. Diff
python3 migrate.py migrate --diff --components slos,workloads

# 4. Import
python3 migrate.py migrate --import-only --components slos,workloads

# 5. Run SLO math-equivalence audit (window + tolerance configured in auditor config)
python3 migrate.py audit-slos
```

SLO auditor output:

```
SLO: Checkout Availability
  NR SLI:  99.94%
  DT SLI:  99.92%
  Delta:    0.02%  PASS
```

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source Dynatrace-NewRelic, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.